

Kirb Stomp (Smash Bros) Game Sprint Requirements

This document details the sprint requirement for our proposed Smash Bros game for our Kirb Stomp team.

Sprint 2 – Game Objects and Sprites

- Create interfaces for different parts of the your game framework, options might include:
 - Noun-like categories
 - Character – 2 player
 - Mario and Link
 - Possibly Kirby or Ganondorf
 - Items / power-ups
 - Blocks for map
 - Projectiles
 - Verb-like categories such as drawables, updateables, collideables, etc.
 - An entity-component design with entities, components, and system managers:
 - AnimationComponent
 - SpriteComponent
 - ProjectileComponent
 - PlayerControlComponent
- Create a class for each object included in the first level of your game
 - Implement drawing of each object
 - Implement movement of each object so that the object moves similar to how it will in the actual game
 - Implement sprite animation
 - Moving animated sprite and attacks
 - Implement state changes of each object.
 - Mario character turn left and right, able to jump and perform some attacks
 - Keep the details of sprite drawing separate from the object's gameplay behavior
- Implement user input for changing the states of objects. In some cases you may use random numbers or timer logic for changing state of non-player objects
 - Command and keymapping (subject to change)
 - **Player controls**
 - Arrow and "wasd" keys should move Mario and change his facing direction.
 - The 'z' and 'n' key should cause character to attack
 - Number keys (1, 2, 3, etc.) should be used to have Mario use a different item (later this will be replaced with a menu system and 'x' and 'm' for the secondary item.

- Use 'e' to cause Mario to become damaged.
- **Block/obstacle controls**
 - Use keys "t" and "y" to cycle between which block is currently being shown (i.e. think of the obstacles as being in a list where the game's current obstacle is being drawn, "t" switches to the previous item and "y" switches to the next)
 - all blocks should be stationary and should not interact with any other objects
- **Item controls**
 - Use keys "u" and "i" to cycle between which item is currently being shown (i.e. think of the items as being in a list where the game's current item is being drawn, "u" switches to the previous item and "i" switches to the next)
 - Items should move and animate as they do in the final game, but should not interact with any other objects
- ~~**Enemy/NPC (other character) controls**~~
 - ~~Use keys "o" and "p" to cycle between which enemy or npc is currently being shown (i.e. think of these characters as being in a list where the game's current character is being drawn, "o" switches to the previous item and "p" switches to the next)~~
 - ~~characters should move, animate, fire projectiles, etc. as they do in the final game, but should not interact with any other objects~~
- **Other controls**
 - Use 'q' to quit and 'r' to reset the program back to its initial state.

Sprint 3 - Level loading and segmentation and collision handling

- Implement collision handling for all types of collisions that can occur, causing state transitions or position changes when necessary.
 - **Attack collisions**
 - Collisions with stage / block
 - Distinguishing jumping to another floating platform vs hitting the block
 - Collisions with items
 - Damage controls
- ~~Instead of placing all objects on screen at once, create the individual "rooms" (or "screens" if you prefer) of the dungeon each with its own subset of objects. Store this information in a file (recommendation: csv or xml) and write code to initialize objects based on the file's contents.~~
 - Changed to implementing 2 maps or moving platforms
 - Test 2 playable characters
 - Different power-ups based on the character or different effects based on map

- ~~• In addition to the regular dungeon, create an artificial level that contains an instance of Link and all types of objects that are found in your first dungeon. For testing purposes, include a way for the user to quickly switch to any room in the dungeon instead of having to walk through multiple rooms to reach it~~
- **Controls and UI**
 - Use the arrow keys and WASD for player movement
 - This sprint is a good time to create a gamepad class, but this is not a hard requirement
 - Continue to allow the use of different items via numeric keys 1, 2, 3, etc.
 - Test all the possible power-ups and attacks for each character
 - ~~○ Recommendation: have all of the item pickups in the starting room, or start the player with an infinite amount of each item~~
 - ~~○ Use the mouse controller as a way to transition between rooms. How you do this is entirely up to you, but consider doing so by clicking on a map. Alternatively, treat the rooms as in an ordered collection, with left click moving to the previous room and right click moving to the next room~~

Sprint 4 - Completed first-level game

- Implement everything needed to play a single game (2 player)
- Update your level file to layout of map – block/platforms location etc.
- Keep the keyboard controls to test each powers up.
- Implement resetting the level and winning screen for the winning player
- Add the Heads Up Display (the font does not need to match the one used in the original game).
- Add sounds. Many machines seem to be buggy about playing sounds in Monogame. Reinstalling DirectX usually fixes this.
- Implement game state transitions:
 - Start Screen
 - The game over screen
 - Pausing
 - The winning game screen
- Implement any other features you deem necessary for the first dungeon. [Instructor's note: the gameplay within the first dungeon of The Legend of Zelda appears to have fewer little interesting details and quirks compared to the gameplay in the first level in Super Mario Bros, so consider implementing a couple of additional items this sprint if you want to really impress us]
- By this portion of the project you should have remove all magic numbers and strings by making utility classes (and optionally inputing their values from a file).
- Begin research on feature ideas for Sprint 5.

Sprint 5 - extending your game framework

- Add AI player
- 2 new character
 - Unlocking new character?
- 1 new map
- Main menu screen
- Winning Animation
- Character selection screen
- Implement switch joy con / controller?